# ALGORITMOS Y ESTRUCTURAS DE DATOS III

El Proceso de resolución de Problemas Complejos
Mediante la aplicación de Estructuras de Datos....
Algunas consideraciones Iniciales….

# Acerca de Problema, Algoritmo y Programa en el Diseño de Software….

Los algoritmos y las estructuras de datos son fundamentales para la resolución de problemas en informática y programación. Se proporciona una visión general de cómo se utilizan para abordar problemas:

Definición de problema: Antes de comenzar a trabajar en la solución de un problema, es crucial comprender completamente el problema en cuestión. Esto implica definir claramente cuál es el objetivo y los requisitos del problema.
Diseño de algoritmo: Una vez que tenga una comprensión sólida del problema, el siguiente paso es diseñar un algoritmo. Un algoritmo es un conjunto de instrucciones paso a paso que describe cómo resolver un problema. Debes considerar la eficiencia y la simplicidad al diseñar el algoritmo.
Selección de estructuras de datos: Las estructuras de datos son herramientas que te permiten organizar y almacenar datos de manera eficiente. La elección de la estructura de datos adecuada es crucial para resolver un problema de manera eficiente. Algunas estructuras de datos comunes incluyen listas, arreglos, colas, pilas, árboles, grafos, etc.
Implementación del algoritmo: Una vez que hayas diseñado el algoritmo y seleccionado las estructuras de datos adecuadas, debes implementar el código que ejecutará el algoritmo. Esto implica traducir el diseño del algoritmo a un lenguaje de programación específico.
Pruebas y depuración: Después de la implementación, debes probar el código para asegurarte de que funcione correctamente. Si encuentras errores o fallos, debes depurar el código para corregirlos.
Optimización: A menudo, es posible optimizar un algoritmo para que sea más rápido o utilice menos recursos. Esto implica ajustar el diseño del algoritmo o la elección de estructuras de datos para mejorar el rendimiento.
Documentación: Es importante documentar tu código de manera adecuada para que otros puedan entenderlo y mantenerlo en el futuro. La documentación también puede ayudarte a ti mismo a recordar cómo funciona tu código.
Evaluación y mejora continua: Después de implementar la solución, es importante evaluar su desempeño y eficacia en la resolución del problema. Si es necesario, puedes realizar mejoras o cambios en el algoritmo o en las estructuras de datos utilizadas.

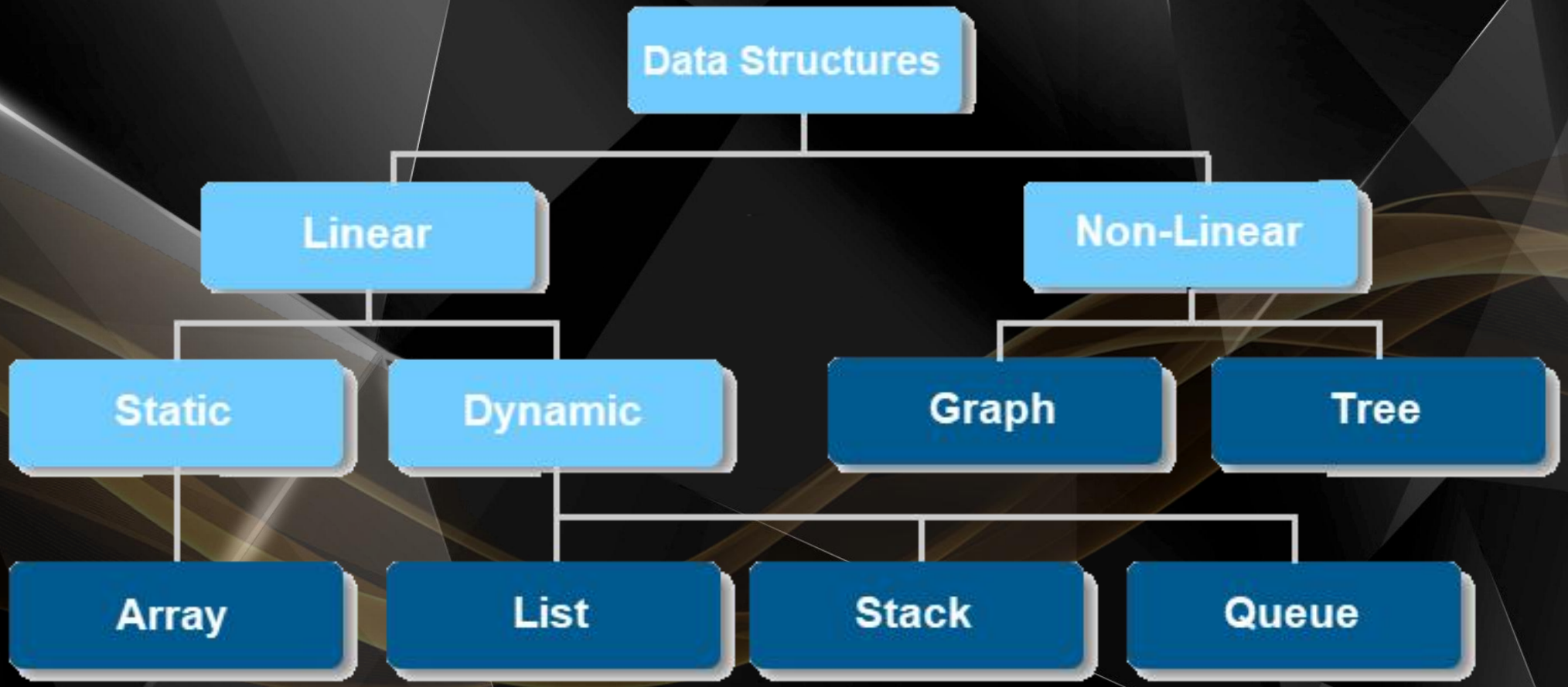
***En resumen, los algoritmos y las estructuras de datos son herramientas esenciales para la resolución de problemas en informática. Un buen diseño de algoritmo y la elección adecuada de estructuras de datos pueden marcar la diferencia en la eficiencia y eficacia de la solución de un problema.*** Además, es importante seguir un proceso sistemático que incluya diseño, implementación, pruebas y mejora continua para resolver problemas de manera efectiva.

# Acerca de las estructuras de Datos (subtipos) y su relación en la resolución de problemas ….

Las estructuras de datos son fundamentales en la programación y se utilizan para resolver una amplia variedad de problemas en diferentes áreas.
se presentan algunas estructuras de datos comunes y ejemplos de los tipos de problemas que pueden ayudarte a resolver:

Arreglos (Arrays) y Matrices:
Problemas de búsqueda y acceso rápido a elementos en una colección de datos.
Algoritmos de ordenamiento, como ordenamiento burbuja, ordenamiento rápido y ordenamiento por mezcla.
Listas enlazadas (Linked Lists):
Implementación de listas o colas con operaciones de inserción y eliminación eficientes.
Problemas que involucran estructuras de datos dinámicas que pueden crecer o reducirse en tamaño.
Pilas (Stacks):
Resolución de problemas relacionados con la gestión de llamadas de funciones (pilas de llamadas).
Evaluación de expresiones matemáticas en notación polaca inversa (postfija).
Colas (Queues):
Implementación de colas de espera para procesamiento en orden de llegada.
Resolución de problemas de búsqueda en amplitud (BFS) en grafos.
Árboles (Trees):
Búsqueda eficiente de elementos en un conjunto de datos (árboles de búsqueda binaria).
Resolución de problemas de estructura jerárquica, como la organización de archivos en un sistema de archivos.
Grafos (Graphs):
Resolución de problemas relacionados con relaciones y conexiones complejas entre objetos.
Algoritmos de búsqueda en grafos, como el algoritmo de Dijkstra o el algoritmo de búsqueda en profundidad (DFS).
Tablas hash (Hash Tables):
Resolución de problemas de búsqueda rápida y eficiente mediante la asignación de claves a valores.
Implementación de diccionarios, bases de datos de clave-valor y cachés.

Estos son solo algunos ejemplos de cómo las estructuras de datos se utilizan para resolver una amplia variedad de problemas en programación. La elección de la estructura de datos adecuada depende de la naturaleza del problema y de los requisitos de rendimiento, por lo que es importante comprender bien las características y ventajas de cada estructura para tomar decisiones informadas al abordar un problema específico.




Un momento por favor....

# U4

# Listas y Listas Enlazadas

Algoritmos y Estructuras de Datos III

# Listas Enlazadas

## Linked List

Algoritmos y Estructuras de Datos III

# U4

## TDA - Listas



CORE

### LISTAS ENLAZADAS

LISTAS VINCULADA

Lista Vinculada (linked list)
Lista Vinculada (linked list) | Insertion
Lista Vinculada (linked list) | Deletion | Head
Lista Vinculada (linked list) | Deletion | End
Lista Vinculada (linked list) | Deletion | Any where
Lista Vinculada (linked list) | Search

L. DOBLE ENLAZ

Listas Doblemente Enlazadas| Introducción.
Lista Doblemente Vinculada (Doubly Linked Lists) | Introducción
Lista Doblemente Vinculada (Doubly Linked Lists) | Insert
Lista Doblemente Vinculada (Doubly Linked Lists) | Delete

Lista Circular (Circular Double Lists) | Introducción
Lista Doble Circular (Circular Double Lists) | Introducción

STL

Listas en la STL| Introducción

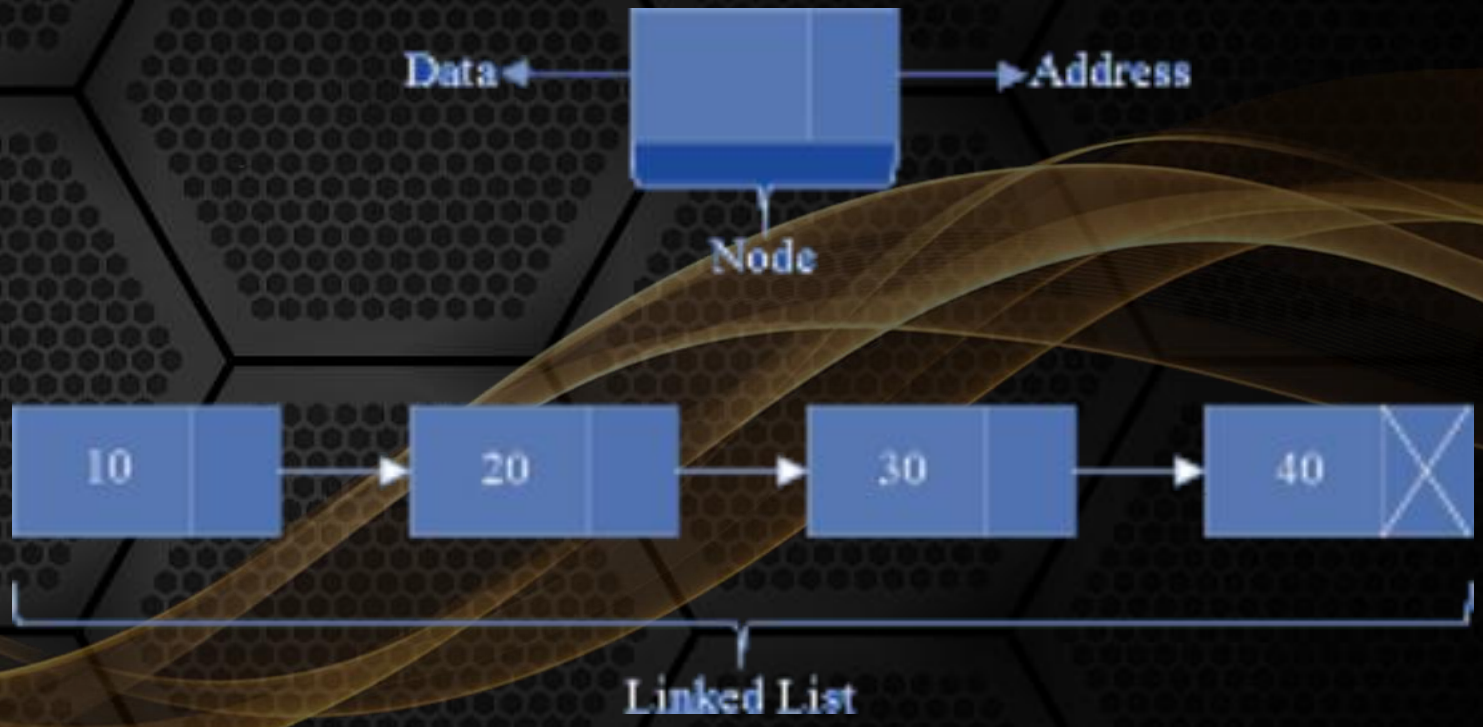
Algoritmos y Estructuras de Datos III

# **Listas Enlazadas|** - Linked Lists

Introducción.

# **Listas Enlazadas|** Introducción.

Linked Lists



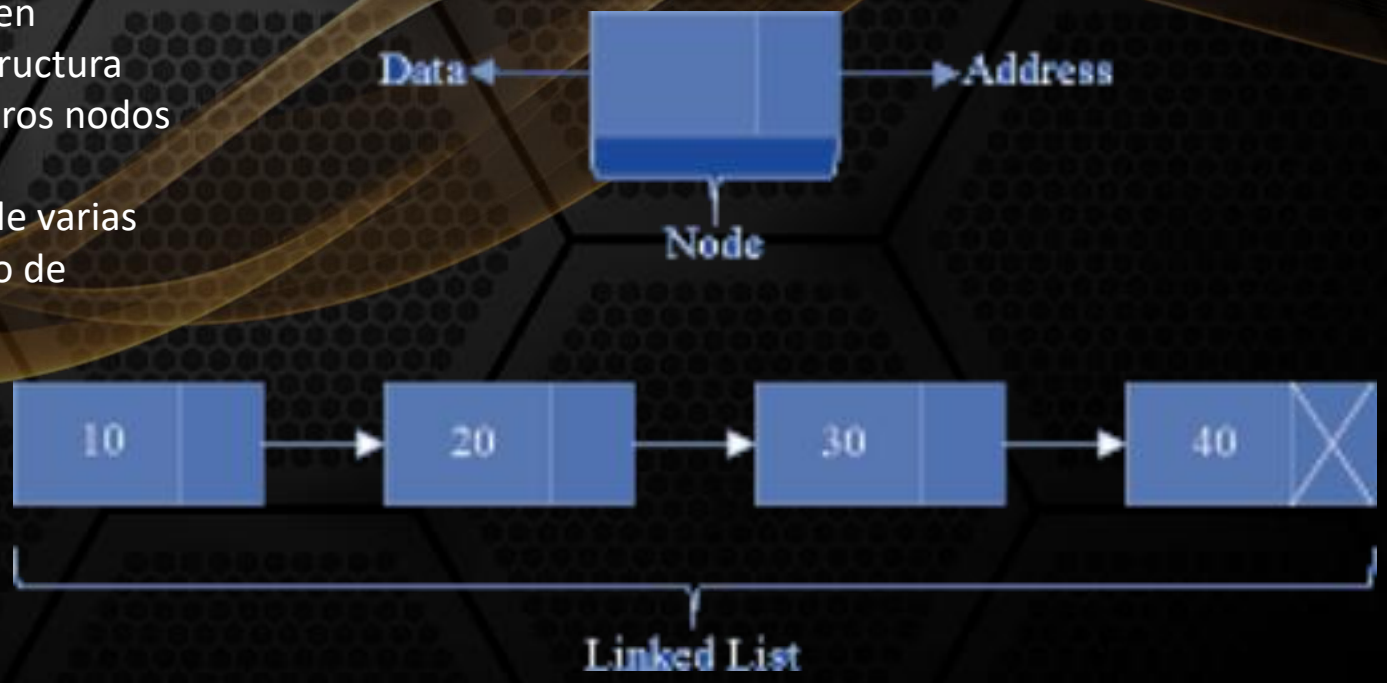
# Listas Enlazadas - Linked Lists

Un array es una estructura de datos muy útil proporcionada en lenguajes de programación. Sin embargo, tiene al menos dos limitaciones:

(1) Su tamaño debe conocerse en el momento de la compilación
(2) Los datos de la matriz están separados en la memoria de la computadora por la misma distancia, lo que significa que insertar un elemento dentro de la matriz requiere cambiar otros datos en esta matriz.

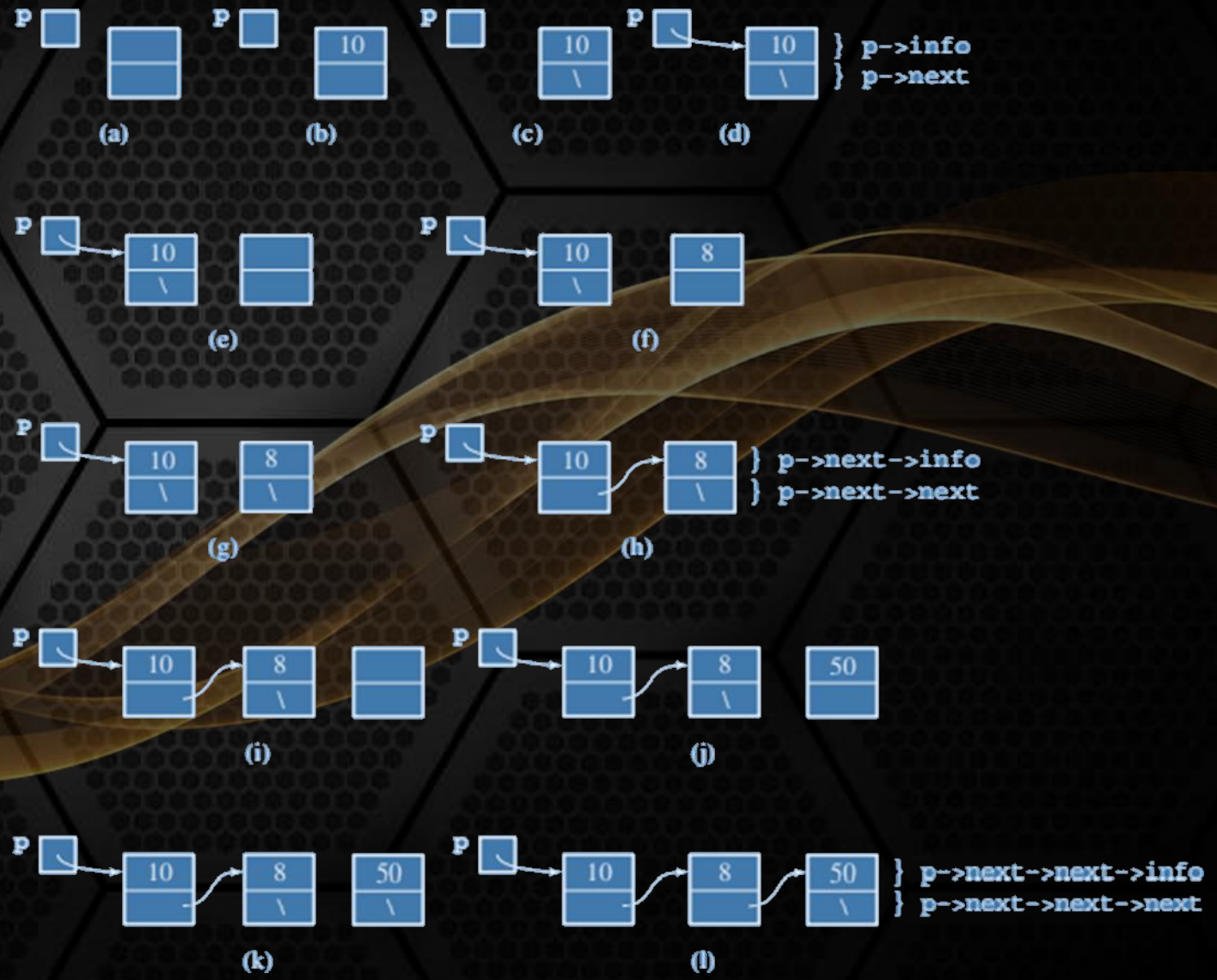
Esta limitación se puede superar mediante el uso de estructuras vinculadas (Linked List).
Una estructura vinculada es una ***colección de nodos*** que almacenan datos y enlaces
a otros nodos. De esta manera, los nodos se pueden ubicar en cualquier lugar de la memoria, y pasar de un nodo de la estructura vinculada a otro se logra almacenando las direcciones de otros nodos en la estructura vinculada.
Aunque las estructuras vinculadas se pueden implementar de varias maneras, la implementación más flexible es mediante el uso de **punteros y clases**.

Adicionalmente implementaremos los Contenedores (Container) que están en la STL,
Programacion Estructirada y Clases.
Utilizando asi las Mejores y Nuevas Capacidades del Lenguaje C++.



# Listas Enlazadas - Linked Lists

Si un nodo contiene un miembro de datos que es un puntero a otro nodo, entonces muchos nodos se pueden unir usando solo una variable para acceder a toda la secuencia de nodos. Tal secuencia de nodos es la implementación más utilizada de una lista vinculada, que es una ***estructura de datos compuesta de nodos***, cada nodo contiene alguna información y un puntero a otro nodo en la lista. Si un nodo tiene un enlace solo a su sucesor en esta secuencia, la lista se denomina lista vinculada por sí sola. Un ejemplo de dicha lista se muestra en la Figura. Tenga en cuenta que sólo se utiliza una variable p para acceder a cualquier nodo de la lista. El último nodo de la lista puede ser reconocido por el puntero nulo. Cada nodo de la lista de la figura 3.1 es una instancia de la siguiente definición de clase:



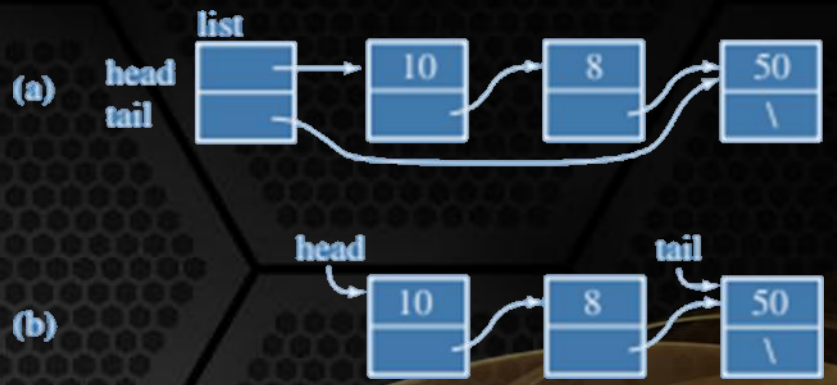

```
class IntSLLNode {
    public:
IntSLLNode() {
next = 0;
}
IntSLLNode(int i, IntSLLNode *in = 0) {
info = i; next = in;
}
int info;
IntSLLNode *next;
};
```

Simple_List.cpp

# Lista Vinculada (linked list)

La implementación de lista vinculada individualmente en la figura utiliza dos clases: una clase, **IntSLLNode**, para los nodos de la lista, y otra, IntSLList, para el acceso a la lista. La clase IntSLList define dos miembros de datos, cabeza y cola, que son punteros al primer y último nodo de una lista. Esto explica por qué todos los miembros de IntSLLNode son declarados públicos.
Debido a que los nodos particulares de la lista son accesibles a través de punteros, los nodos se hacen inaccesibles para objetos externos al declarar que la cabeza y la cola son privadas para que el principio de ocultación de información no se vea realmente comprometido. Si algunos de los miembros de IntSLLNode fueron declarados no públicos, entonces las clases derivadas de IntSLList no pudieron acceder a ellos.

En la figura se muestra un ejemplo de una lista. La lista se declara con la instrucción IntSLList list;




Además de los miembros head y tail, la clase IntSLList también define funciones de miembro que nos permiten manipular las listas.

Ahora observamos más de cerca algunas operaciones básicas en listas vinculadas.

# Listas Enlazadas| Inserción

(linked list) | Insertion

# Lista Vinculada (linked list) | Insertion

Adición de un nodo al **principio**

La adición de un nodo al **principio** de una lista vinculada se realiza en cuatro pasos.

1. Se crea un nodo vacío. Está vacío en el sentido de que el programa que realiza la inserción no asigna ningún valor a los miembros de datos del nodo (Figura a).
2. El miembro info del nodo se inicializa en un entero en particular (Figura b).
3. Debido a que el nodo se incluye al frente de la lista, el siguiente miembro se convierte en un puntero al primer nodo de la lista; es decir, el valor actual de la cabeza (Figura c).
4. El nuevo nodo precede a todos los nodos de la lista, pero este hecho debe reflejarse en el valor de head; de lo contrario, no se podrá acceder al nuevo nodo. Por lo tanto, el cabezal se actualiza para convertirse en el puntero al nuevo nodo (Figura ).




Los cuatro pasos son ejecutados por la función miembro addToHead(). La función ejecuta los tres primeros pasos indirectamente llamando al constructor IntSLLNode (el,head). El último paso se ejecuta directamente en la función asignando la dirección del nodo recién creado a head. La función miembro addToHead() señala un caso especial, a saber, insertar un nuevo nodo en una lista vinculada vacía. En una lista vinculada vacía, tanto la cabeza como la cola son nulas; por lo tanto, ambos se convierten en punteros al único nodo de la nueva lista. Al insertar en una lista existente, solo es necesario actualizar el cabezal.

# Lista Vinculada (linked list) | Insertion

Nuevo nodo al final de la lista

El proceso de agregar un nuevo nodo al final de la lista tiene cinco pasos.

1. Se crea un nodo vacío (Figura a).
2. El miembro info del nodo se inicializa en un entero el (Figura b).
3. Debido a que el nodo se incluye al final de la lista, el siguiente miembro se establece en null (Figura c).
4. El nodo ahora se incluye en la lista haciendo que el siguiente miembro del último nodo de la lista sea un puntero al nodo recién creado (Figura d).
5. El nuevo nodo sigue a todos los nodos de la lista, pero este hecho debe reflejarse en el valor de tail, que ahora se convierte en el puntero del nuevo nodo (Figura e).

Todos estos pasos se ejecutan en la cláusula if de addToTail().
La cláusula else de esta función sólo se ejecuta si la lista vinculada está vacía.

Si no se incluyera este caso, el programa puede bloquearse porque en la cláusula if hacemos una asignación al siguiente miembro del nodo referido por tail. En el caso de una lista vinculada vacía, es un puntero a un miembro de datos no vigente de un nodo no existe.



# Lista Vinculada (linked list) | Insertion

## conclusion| Insertion

El proceso de insertar un nuevo nodo al principio de la lista es muy similar al proceso de insertar un nodo al final de la lista. Esto se debe a que la implementación de IntSLList utiliza dos miembros punteros: cabeza y cola. Por esta razón, tanto addToHead() como addToTail() se pueden ejecutar en tiempo constante O(1); es decir, independientemente del número de nodos de la lista, el número de operaciones realizadas por estas dos funciones miembro no supera algún número constante c.

Tenga en cuenta que debido a que el puntero de cabeza nos permite tener acceso a una lista vinculada, el puntero de cola no es indispensable; su única función es tener acceso inmediato al último nodo de la lista. Con este acceso, se puede agregar fácilmente un nuevo nodo al final de la lista. Si no se usara el puntero de cola, entonces agregar un nodo al final de la lista sería más complicado porque primero tendríamos que llegar al último nodo para adjuntarle un nuevo nodo. Esto requiere escanear la lista y requiere pasos O(n) para finalizar; es decir, es linealmente proporcional a la longitud de la lista.

El proceso de escaneo de listas se ilustra cuando
se discute la eliminación del último nodo

# Listas Enlazadas| Borrado

(linked list) | Deletion

# Lista Vinculada (linked list) | Deletion Head

Una operación de **eliminación** consiste en eliminar un nodo al principio de la lista y devolver el valor almacenado en él. Esta operación es implementada por la función miembro **deleteFromHead()**.
En esta operación, la información del primer nodo se almacena temporalmente en una variable local , y luego se restablece el cabezal para que lo que era el  segundo nodo se convierte en el primer nodo.
De esta manera, el primer nodo anterior se puede eliminar en tiempo constante O(1) (Ver Figura)




**A diferencia de antes, ahora hay dos casos especiales a considerar.**
Un caso es cuando intentamos eliminar un nodo de una lista vinculada vacía. Si se hace tal intento, es muy probable que el programa se bloquee, lo que no queremos que suceda. La persona que llama también debe saber que se hace tal intento de realizar una determinada acción. Después de todo, si la persona que llama espera que se devuelva un número de la llamada a deleteFromHead() y no se puede devolver ningún número, es posible que la persona que llama no pueda realizar algunas otras operaciones. Hay varias maneras de abordar este problema.

Una forma es usar una declaración de afirmación y otra arrojado una excepción.

# (Patentices Metodológicos) Macros en C++

En C++, una macro es una secuencia de comandos o código preprocesado que se utiliza para realizar sustituciones de texto en el código fuente antes de que el compilador realice su trabajo. Las macros se definen utilizando la directiva #define. Puedes usar macros para crear funciones simples o realizar reemplazos de texto.
Aquí tienes un ejemplo básico de una macro que funciona como una función simple:

```
#include <iostream>
#define SUMA(a, b) ((a) + (b))

 int main()
{
   int x = 5;
   int y = 3;
   int resultado = SUMA(x, y);
   std::cout << "La suma es: " << resultado << std::endl;
   return 0;
}
```

En este ejemplo, la macro SUMA toma dos argumentos a y b y devuelve su suma. Cuando compilas este código, el preprocesador reemplaza todas las ocurrencias de SUMA(x, y) con (x) + (y), y luego el compilador continúa con el proceso de compilación.
Las macros pueden ser útiles para tareas simples, pero debes tener cuidado al usarlas, ya que pueden llevar a problemas de legibilidad y mantenimiento del código. En C++, se recomienda el uso de funciones en lugar de macros siempre que sea posible, ya que las funciones son más seguras y más fáciles de depurar.

# (Patentices Metodológicos) Assert en C++

En C++, assert es una macro que se utiliza para verificar una condición en tiempo de ejecución y detener el programa si la condición es falsa. La macro assert se define en la biblioteca de la cabecera <cassert>. Aquí tienes un ejemplo de cómo se usa:

```
#include <cassert>
 int main()
{
        int x = 5;
       assert(x == 10);
        // Esto provocará un error en tiempo de ejecución ya que la condición es falsa
       return 0;
}
```

En este ejemplo, la condición x == 10 es falsa, por lo que la macro assert detendrá el programa y mostrará un mensaje de error en la salida estándar. Es importante mencionar que assert es una herramienta útil para detectar errores en el código durante el desarrollo y las pruebas, pero se debe tener cuidado al usarlo en código de producción, ya que puede desactivarse en compilaciones de liberación. Además, assert se basa en la evaluación de una condición y no debe usarse para manejar errores de manera adecuada en una aplicación.

# Lista Vinculada (linked list) | Deletion | Head

Una forma es usar una declaración assert

```
int IntSLList::deleteFromHead() {
 assert(!isEmpty());
 // terminate the program if false;
 int el = head->info;
 . . . . . .
 return el;
     }
```

La instrucción assert comprueba la condición !isEmpty(), y si la condición es false, el programa se anula. Esta es una solución cruda porque la persona que llama puede desear continuar incluso si no se devuelve ningún número de deleteFromHead().

Otra solución es lanzar una excepción y atraparla por parte del usuario, como en:

```
int IntSLList::deleteFromHead() {
if (isEmpty())
throw("Empty");
int el = head->info;
. . . . . .
return el;
    }
```

Se espera que la cláusula throw con el argumento string tenga una cláusula trycatch coincidente en el llamador (o el llamador del llamador, etc.) también con el argumento string, que atrapa la excepción, como en

```
 void f() {
 . . . . . .
 try {
 n = list.deleteFromHead();
 // do something with n;
 } catch(char *s) {
 cerr << “Error: “ << s << endl;
 }
 . . . . . .
     }
```

# Lista Vinculada (linked list) | **Deletion | Head**

Esta solución le da a la persona que llama cierto control sobre la situación anormal sin hacerla letal para el programa como con el uso de la declaración assert. El usuario es responsable de proporcionar un controlador de excepciones en forma de la instrucción **try-catch**, con la solución adecuada al caso particular. Si no se proporciona la instrucción, el programa se bloquea cuando se lanza la excepción. La función f() sólo puede imprimir un mensaje de que una lista está vacía cuando se intenta eliminar un número de una lista vacía, otra función g() puede asignar un cierto valor a n en tal caso, y otra función h() puede encontrar tal situación perjudicial para el programa y abortar el programa por completo.

La idea de que el usuario es responsable de proporcionar una acción en el caso de una excepción también se presume en la implementación dada en el código mostrado (a Figura) La función miembro asume que la lista no está vacía. Para evitar que el programa se bloquee, la función miembro isEmpty() se agrega a la clase IntSLList y el usuario debe usarla como en:

```
if (!list.isEmpty())
n = list.deleteFromHead();
    else do not delete;
```

Tenga en cuenta que incluir una instrucción if similar en deleteFromHead() no resuelve el problema.
Considere este código:

```
int IntSLList::deleteFromHead() {
if (!isEmpty()) { // if nonempty list;
int el = head->info;
. . . . . .
return el;
}
else return 0;
    }
```

# Lista Vinculada (linked list) | **Deletion | Head**

Si se agrega una instrucción if, también se debe agregar la cláusula else; de lo contrario, el programa no compila porque "no todas las rutas de control devuelven un valor". Pero ahora, si se devuelve 0, la persona que llama no sabe si el 0 devuelto es un signo de error o si es un 0 literal recuperado de la lista. Para evitar cualquier confusión, la persona que llama debe usar un if para probar si la lista está vacía antes de llamar a deleteFromHead(). De esta manera, una declaración if sería redundante. Para mantener la uniformidad en la interpretación del valor devuelto, la última solución se puede modificar de modo que en lugar de devolver un entero, la función devuelva el puntero a un entero:

```
int* IntSLList::deleteFromHead() {
if (!isEmpty()) { // if nonempty list;
int *el = new int(head->info);
. . . . . .
return el;
}
else return 0;
    }
```

donde 0 en la cláusula else es el puntero nulo, no el número 0. En este caso, la llamada a la función  n = *list.deleteFromHead();  da como resultado un bloqueo del programa si deleteFromHead() devuelve el puntero nulo. Por lo tanto, el autor de la llamada debe realizar una prueba antes de llamar a deleteFromHead() para comprobar si la lista está vacía o si se debe usar una variable de puntero,
 int *p = list.deleteFromHead();  y luego se realiza una prueba después de la llamada para verificar si p es nulo. En cualquier caso, esto significa que la instrucción if en deleteFromHead() es redundante. El segundo caso especial es cuando la lista solo tiene un nodo que eliminar. En este caso, la lista se vacía, lo que requiere establecer la cola y la cabeza en null.

# Lista Vinculada (linked list) | **Deletion | End**

La segunda operación de eliminación consiste en eliminar un nodo del final de la lista, y se implementa como la función miembro deleteFromTail(). El problema es que después de eliminar un nodo, la cola debe referirse a la nueva cola de la lista; es decir, la cola tiene que ser movida hacia atrás por un nodo. Pero retroceder es imposible porque no hay un enlace directo desde el último nodo a su predecesor. Por lo tanto, este predecesor debe encontrarse buscando desde el principio de la lista y deteniéndose justo antes de la cola. Esto se logra con una variable temporal tmp que escanea la lista dentro del bucle for. La variable tmp se inicializa en el cabezal de la lista y, a continuación, en cada iteración del bucle, se avanza al siguiente nodo. Si la lista es como en la Figura a, entonces tmp primero se refiere al nodo principal que contiene el número 6; después de ejecutar la asignación tmp = tmp->siguiente, tmp se refiere al segundo nodo (Figura b). Después de la segunda iteración y ejecutando la misma asignación, tmp se refiere al tercer nodo (Figura c). Debido a que este nodo es también el siguiente al último nodo, se sale del bucle, después de lo cual se elimina el último nodo (Figura d). Debido a que tail ahora apunta a un nodo no existe, se establece inmediatamente para apuntar al siguiente al último nodo al que actualmente apunta tmp (Figura e). Para marcar el hecho de que es el último nodo de la lista, el siguiente miembro de este nodo se establece en null (Figura f).



# Lista Vinculada (linked list) | **Deletion | End**

Tenga en cuenta que en el bucle for, se utiliza una variable temporal para analizar la lista. Si el bucle se simplificara a:

```
for ( ; head->next != tail; head = head->next);
```

luego, la lista se escanea solo una vez y se pierde el acceso al principio de la lista porque la cabeza se actualizó permanentemente al nodo siguiente al último, que está a punto de convertirse en el último nodo.
Es absolutamente crítico que, en casos como este, se utilice una variable temporal para que el acceso al principio de la lista se mantenga intacto. Al quitar el último nodo, los dos casos especiales son los mismos que en deleteFromHead(). Si la lista está vacía, entonces no se puede eliminar nada, pero lo que se debe hacer en este caso se decide en el programa de usuario al igual que en el caso de deleteFromHead(). El segundo caso es cuando una lista de un solo nodo se vacía después de eliminar su único nodo, lo que también requiere establecer la cabeza y la cola en null. La parte que más tiempo consume de deleteFromTail() es encontrar el siguiente al último nodo realizado por el bucle for. Está claro que el bucle realiza n – 1 iteraciones en una lista de n nodos, que es la razón principal por la que esta función miembro tarda O(n) en eliminar el último nodo.



# Lista Vinculada (linked list) | Deletion | Any where

Las dos operaciones de eliminación discutidas eliminan un nodo de la cabeza o de la cola (es decir, siempre desde la misma posición) y devuelven el entero que se encuentra en el nodo que se está eliminando. Un enfoque diferente es cuando queremos eliminar un nodo que contiene un entero particular independientemente de la posición de este nodo en la lista.

Estos pueden estar al principio, al final o en cualquier lugar dentro de la lista. Brevemente, un nodo tiene
ubicarse primero y luego separarse de la lista vinculando el predecesor de este
nodo directamente a su sucesor. Debido a que no sabemos dónde puede estar el nodo, el
El proceso de encontrar y eliminar un nodo con un determinado número entero es mucho más complejo.
que las operaciones de eliminación discutidas hasta ahora.
La función miembro deleteNode ()
(Figura 3.2) es una implementación de este proceso

```
void IntSLList::deleteNode(int el)
{
    if (head != 0) // if nonempty list;
        if (head == tail && el == head->info)
        {                   // if only one
            delete head; // node in the list;
            head = tail = 0;
        }
        else if (el == head->info)
        { // if more than one node in the list
            IntSLLNode *tmp = head;
            head = head->next;
            delete tmp; // and old head is deleted;
        }
        else
        { // if more than one node in the list
            IntSLLNode *pred, *tmp;
            for (pred = head, tmp = head->next;  // and a nonhead node
                 tmp != 0 && !(tmp->info == el); // is deleted;
                 pred = pred->next, tmp = tmp->next)
                ;
            if (tmp != 0)
            {
                pred->next = tmp->next;
                if (tmp == tail)
                    tail = pred;
                delete tmp;
            }
        }
}
```

# Lista Vinculada (linked list) | **Deletion** | Any where

En el párrafo anterior se examina sólo un caso. Estos son los casos restantes:
1. Un intento de eliminar un nodo de una lista vacía, en cuyo caso la función se cierra inmediatamente.
2. Eliminación del único nodo de una lista vinculada de un nodo: tanto la cabeza como la cola se establecen en null
3. Eliminar el primer nodo de la lista con al menos dos nodos, lo que requiere actualizar el encabezado.
4. Eliminar el último nodo de la lista con al menos dos nodos, lo que lleva a la actualización de la cola.
5. Un intento de eliminar un nodo con un número que no está en la lista: no hacer nada.

Está claro que el mejor caso para deleteNode() es cuando se va a eliminar el nodo principal, lo que lleva O(1) tiempo para lograrlo. El peor de los casos es cuando es necesario eliminar el último nodo, lo que reduce deleteNode() a deleteFromTail() ya su rendimiento O(n). ¿Cuál es el caso promedio? Depende de cuántas iteraciones ejecute el bucle for. Suponiendo que cualquier nodo de la lista tiene las mismas posibilidades de ser eliminado, el ciclo no realiza ninguna iteración si es el primer nodo, una iteración si es el segundo nodo, . . . , y finalmente n – 1 iteraciones si es el último nodo. Para una larga secuencia de deleciones, una deleción requiere en promedio

$$\frac{0+1+\ldots+(n-1)}{n} = \frac{\frac{(n-1)n}{2}}{n} = \frac{n-1}{2}$$

Es decir, en promedio, deleteNode() ejecuta pasos O(n) para finalizar, al igual que en el peor de los casos.



# Lista Vinculada (linked list) | **Search**

Las operaciones de inserción y eliminación modifican las listas vinculadas. La operación de búsqueda analiza una lista existente para saber si hay un número en ella. Implementamos esta operación con la función miembro booleana isInList(). La función utiliza una variable temporal tmp para recorrer la lista a partir del nodo principal. El número almacenado en cada nodo se compara con el número que se busca, y si los dos números son iguales, se sale el bucle; de lo contrario, tmp se actualiza a tmp->next para que se pueda investigar el siguiente nodo. Después de llegar al último nodo y ejecutar la asignación tmp = tmp->next, tmp se convierte en null, que se utiliza como una indicación de que el número el no está en la lista. Es decir, si tmp no es nulo, la búsqueda se suspendió en algún lugar dentro de la lista porque se encontró el. Es por eso que isInList() devuelve el resultado de la comparación tmp != 0: si tmp no es null, se encontró el y se devuelve true. Si tmp es null, la búsqueda no se realizó correctamente y se devuelve false. Con un razonamiento similar al utilizado para determinar la eficiencia de deleteNode(), isInList() toma O(1) tiempo en el mejor de los casos y O(n) en el peor y promedio de los casos. En la discusión anterior, se han enfatizado las operaciones en nodos. Sin embargo, una lista vinculada se construye por el bien de almacenar y procesar información, no por el bien de sí misma. Por lo tanto, el enfoque utilizado en esta sección es limitado en el que la lista sólo puede almacenar enteros. Si quisiéramos una lista vinculada para números flotantes o para matrices de números, entonces se tendría que declarar una nueva clase con un nuevo conjunto de funciones miembro, todas ellas parecidas a las discutidas aquí. Sin embargo, es más ventajoso declarar dicha clase solo una vez sin decidir de antemano qué tipo de datos se almacenarán en ella. Esto se puede hacer muy convenientemente en C ++ con plantillas.

```
bool IntSLList::isInList(int el) const
{
    IntSLLNode *tmp;
    for (tmp = head; tmp != 0 && !(tmp-
>info == el); tmp = tmp->next)
        ;
    return tmp != 0;
}
```

# **Listas Enlazadas|** - Linked Lists

Lista Vinculada (linked list) | **Insertion**| **Begin**
Lista Vinculada (linked list) | **Insertion**| **End**
Lista Vinculada (linked list) | **Deletion** | **Head**
Lista Vinculada (linked list) | **Deletion** | **End**
Lista Vinculada (linked list) | **Deletion** | **Any where**
Lista Vinculada (linked list) | **Search**




(Patentices Metodológicos) Macros en C++

(Patentices Metodológicos) Assert en C++

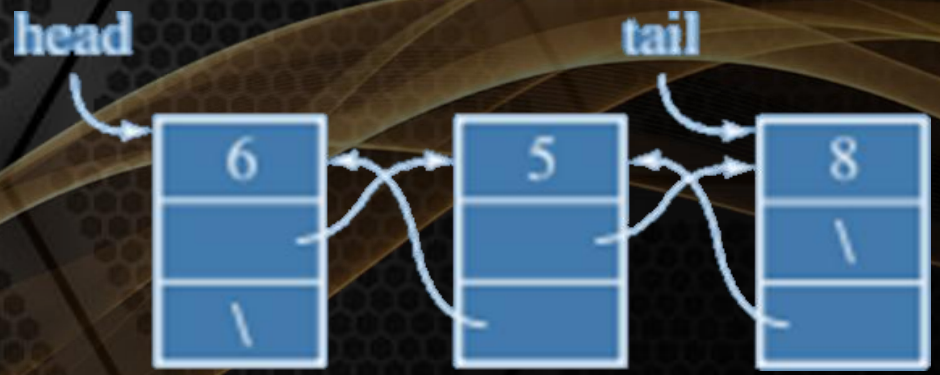
# Listas Doblemente Enlazadas | Introducción.

Doubly Linked Lists

# Lista Doblemente Vinculada (**Doubly Linked Lists**) | Introducción

La función miembro deleteFromTail() indica un problema inherente a las listas enlazadas individualmente. Los nodos de dichas listas contienen sólo punteros a los sucesores; por lo tanto, no hay acceso inmediato a los predecesores. Por esta razón, deleteFromTail() se implementó con un bucle que nos permitió encontrar el predecesor de tail. Aunque este predecesor está, por así decirlo, a la vista, está fuera de su alcance. Tenemos que escanear toda la lista para detenernos justo en frente de la cola para eliminarla. Para listas largas y para ejecuciones frecuentes de deleteFromTail(), esto puede ser un impedimento para el procesamiento rápido de listas.

***Para evitar este problema, la lista vinculada se redefine de modo que cada nodo de la lista tenga dos punteros, uno al sucesor y otro al predecesor. Una lista de este tipo se denomina lista doblemente vinculada y se ilustra en la figura***

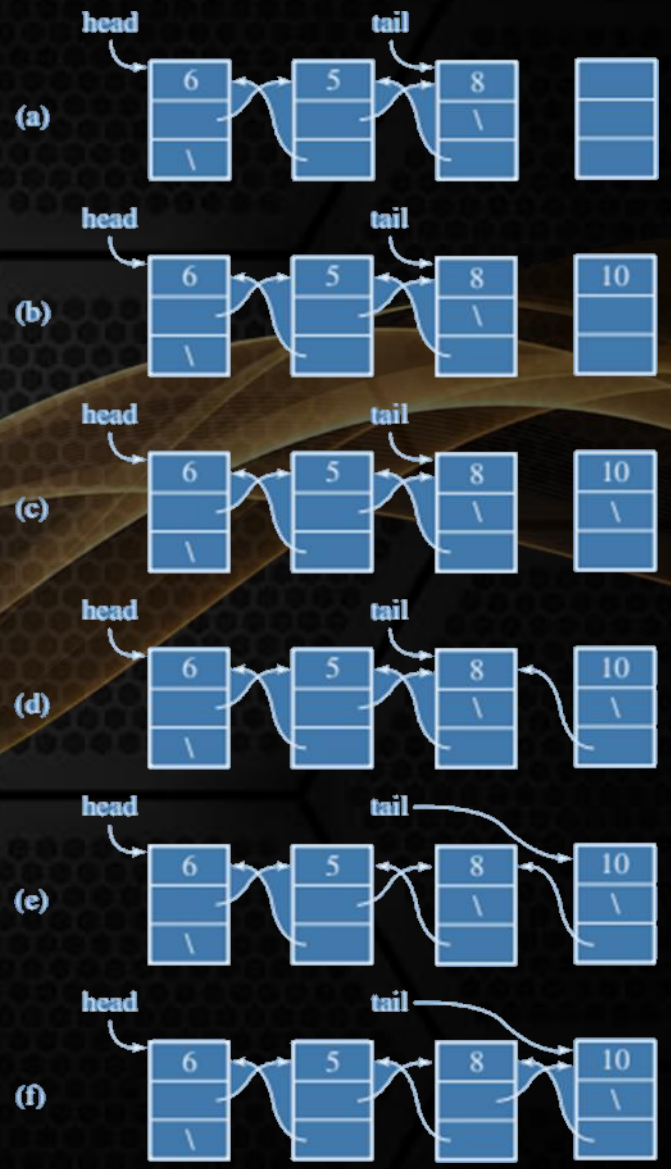


# Lista Doblemente Vinculada (**Doubly Linked Lists**) | Introducción

Las funciones de los miembros para procesar listas doblemente vinculadas son un poco más complicadas que sus contrapartes vinculadas individualmente porque hay un miembro puntero más que debe mantenerse.
Sólo se discuten dos funciones: una función para insertar un nodo al final de la lista doblemente vinculada y una función para eliminar un nodo. Para agregar un nodo a una lista, el nodo debe crearse, sus miembros de datos se inicializaron correctamente y, a continuación, el nodo debe incorporarse a la lista. La inserción de un nodo al final de una lista doblemente vinculada realizada por addToDLLTail() se ilustra en la figura.

El proceso se realiza en seis pasos:

1. Se crea un nuevo nodo (Figuraa), y luego se inicializan sus tres miembros de datos:
2. el miembro info al número el que se está insertando (Figura b),
3. el siguiente miembro a null (Figura c),
4. y el miembro anterior al valor de tail para que este miembro apunte al último nodo de la lista (Figura d). Pero ahora, el nuevo nodo debería convertirse en el último nodo; por lo tanto,
5. tail se establece para apuntar al nuevo nodo (Figura e). Pero el nuevo nodo aún no es accesible desde su predecesor; para rectificar esto,
6. el siguiente miembro del predecesor está configurado para apuntar al nuevo nodo (Code Figura f).



# Lista Doblemente Vinculada (**Doubly Linked Lists**) | Introducción

Un caso especial se refiere al último paso. En este paso se supone que el nodo recién creado tiene un predecesor, por lo que accede a su miembro anterior. Debería ser obvio que para una lista vinculada vacía, el nuevo nodo es el único nodo de la lista y no tiene predecesor. En este caso, tanto la cabeza como la cola se refieren a este nodo, y el sexto paso ahora es establecer la cabeza para apuntar a este nodo. Tenga en cuenta que el paso cuatro (establecer el miembro anterior en el valor de tail) se ejecuta correctamente porque para una lista inicialmente vacía, tail es null. Por lo tanto, null se convierte en el valor del miembro anterior del nuevo nodo. Eliminar el último nodo de la lista doblemente vinculada es sencillo porque hay acceso directo desde el último nodo a su predecesor, y no se necesita ningún bucle para eliminar el último nodo. Al eliminar el último nodo de la lista de la figura a, la variable temporal el se establece en el valor del nodo, luego tail se establece en su predecesor (figura b) y el último y ahora redundante nodo se elimina (figura c). De esta manera, el último nodo se convierte en el último nodo. El siguiente miembro del nodo de cola es una referencia colgante; por lo tanto, next se establece en null (Figura d). El último paso es devolver la copia del objeto almacenado en el nodo eliminado



# Lista Doblemente Vinculada (**Doubly Linked Lists**) | Introducción

Un intento de eliminar un nodo de una lista vacía puede provocar un bloqueo del programa. Por lo tanto, el usuario tiene que comprobar si la lista no está vacía antes de intentar eliminar el último nodo. Al igual que con deleteFromHead() de la lista enlazada por uno, el autor de la llamada debe tener una instrucción if

```
if (!list.isEmpty())
n = list.deleteFromDLLTail();
    else do not delete;
```

Otro caso especial es la eliminación del nodo de una lista vinculada de un solo nodo. En este caso, tanto la cabeza como la cola se establecen en null. Debido a la accesibilidad inmediata del último nodo, tanto addToDLLTail() como deleteFromDLLTail() se ejecutan en tiempo constante O(1). Las funciones para operar al principio de la lista doblemente vinculada se obtienen fácilmente de las dos funciones que acabamos de discutir cambiando de cabeza a cola y viceversa, cambiando junto a prev y viceversa, e intercambiando el orden de los parámetros al ejecutar nuevo.

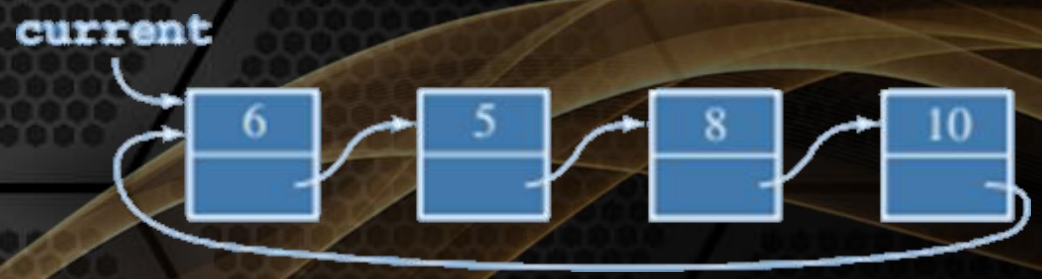


# Listas Circulares | Introducción.

Circular Lists

# Lista Circular (Circular **Lists**) | Introducción

En algunas situaciones, se necesita una lista circular en la que los nodos formen un anillo: la lista es finita y cada nodo tiene un sucesor. Un ejemplo de tal situación es cuando varios procesos utilizan el mismo recurso durante la misma cantidad de tiempo, y tenemos que asegurarnos de que cada proceso tenga una parte justa del recurso. Por lo tanto, todos los procesos (que sus números sean 6, 5, 8 y 10, como en la Figura se colocan en una lista circular accesible a través de la corriente del puntero. Después de acceder a un nodo de la lista y recuperar el número de proceso del nodo para activar este proceso, la corriente se mueve al siguiente nodo para que el siguiente proceso se pueda activar la próxima vez



# Lista Circular (Circular **Lists**) | Introducción

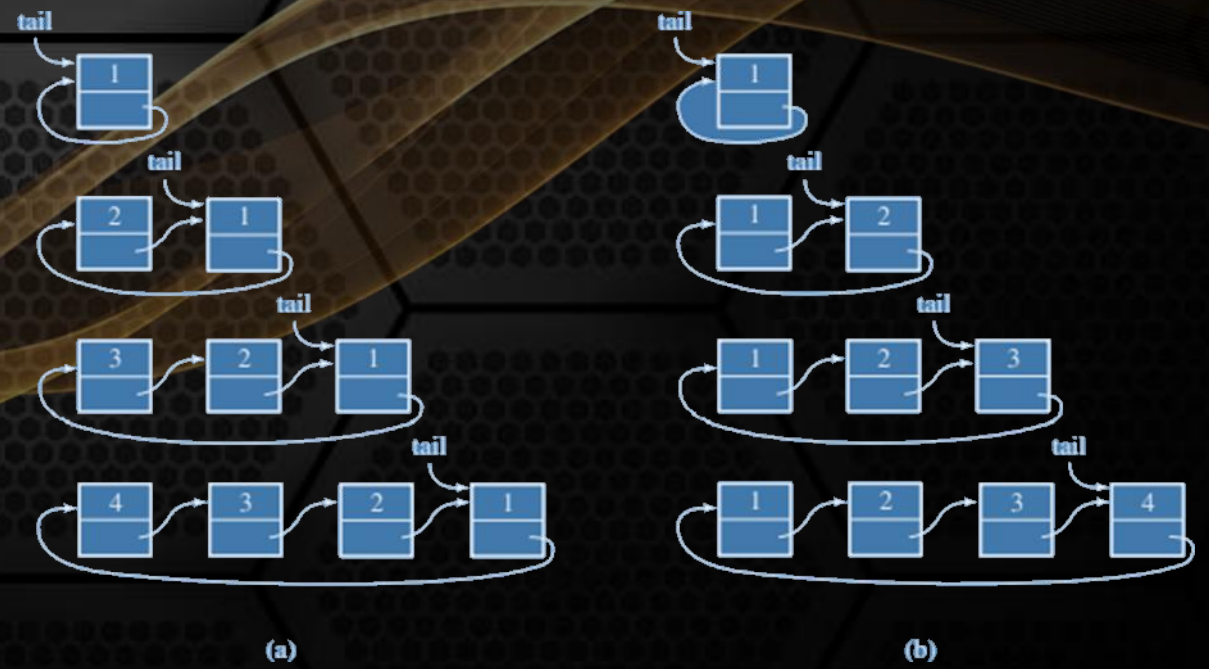
En una implementación de una lista circular enlazada individualmente, podemos usar solo un puntero permanente, cola, a la lista a pesar de que las operaciones en la lista requieren acceso a la cola y su sucesor, la cabeza. Con ese fin, una lista lineal enlazada por sí sola, como vemos, utiliza dos punteros permanentes, cabeza y cola. La figura (a) muestra una secuencia de inserciones en la parte delantera de la lista circular, y la figura (b) ilustra las inserciones al final de la lista. Como ejemplo de una función miembro que opera en dicha lista, presentamos una función para insertar un nodo en la cola de una lista circular enlazada individualmente en O(1):

```
void addToTail(int el)
{
     if  (isEmpty())
     {
          tail = new IntSLLNode(el);
          tail->next = tail;

     }
      else
     {
          tail->next = new IntSLLNode(el, tail->next);
          tail = tail->next;

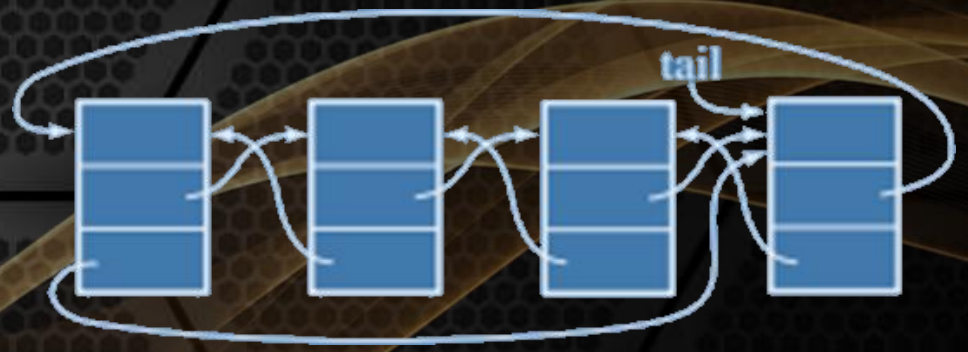
     }

}
```



# Lista Doble Circular (Circular **Double Lists**) | Introducción

La implementación que se acaba de presentar no está exenta de problemas. Una función miembro para la eliminación del nodo tail requiere un bucle para que tail se pueda establecer en su predecesor después de eliminar el nodo. Esto hace que esta función elimine el nodo de cola en tiempo O(n). Además, el procesamiento de datos en orden inverso (impresión, búsqueda, etc.) no es muy eficiente. Para evitar el problema y aún así poder insertar y eliminar nodos en la parte delantera y al final de la lista sin usar un bucle, se puede usar una lista circular doblemente vinculada. La lista forma dos anillos: uno que avanza a través de los siguientes miembros y otro que retrocede a través de los miembros anteriores. La figura ilustra dicha lista accesible a través del último nodo. La eliminación del nodo del final de la lista se puede hacer fácilmente porque hay acceso directo al último nodo que debe actualizarse en el caso de dicha eliminación. En esta lista, tanto la inserción como la eliminación del nodo cola se pueden hacer en tiempo O(1)



# Listas en la STL| Introducción.

Lists in the Standard Template Library

# **Listas en la STL** ( Lists in STL) | Introducción

El contenedor de secuencia de lista es una implementación de varias operaciones en los nodos de una lista vinculada. El STL implementa una lista como una lista genérica doblemente vinculada con punteros a la cabeza y a la cola.

La lista de clases sólo se puede utilizar en un programa si se incluye con la instrucción **#include**
Se genera una nueva lista con la lista de instrucciones :

```
#include <list>
list<T> lst;
```

Donde T puede ser cualquier tipo de datos. Si se trata de un tipo definido por el usuario, el tipo también debe incluir un constructor predeterminado, que es necesario para la inicialización de nuevos nodos. De lo contrario, el compilador no puede compilar las funciones miembro con argumentos inicializados por el constructor predeterminado. Estos incluyen un constructor y funciones resize(), assign(), y una versión de insert().
El contenedor vectorial solo tiene tres funciones miembro que no se encuentran en el contenedor de lista (at(), capacity() y reserve()), pero hay varias funciones miembro de lista que no se encuentran en el contenedor vectorial.

# **Listas en la STL** ( Lists in STL) | Introducción

std::lista
template < class T, class Alloc = allocator<T> > class list;

Las listas son contenedores de secuencia que permiten operaciones de inserción y borrado de tiempo constante en cualquier lugar dentro de la secuencia, y la iteración en ambas direcciones.

Los contenedores de listas se implementan como listas doblemente vinculadas; Las listas doblemente vinculadas pueden almacenar cada uno de los elementos que contienen en ubicaciones de almacenamiento diferentes y no relacionadas. El orden se mantiene internamente mediante la asociación a cada elemento de un enlace al elemento que lo precede y un enlace al elemento que lo sigue.

Son muy similares a forward_list: la principal diferencia es que forward_list objetos son listas de un solo enlace, y por lo tanto solo se pueden iterar hacia adelante, a cambio de ser algo más pequeños y eficientes.

En comparación con otros contenedores de secuencia estándar base (matriz, vector y deque), las listas generalmente funcionan mejor al insertar, extraer y mover elementos en cualquier posición dentro del contenedor para el que ya se ha obtenido un iterador y, por lo tanto, también en algoritmos que hacen un uso intensivo de estos, como los algoritmos de clasificación.

El principal inconveniente de las listas y forward_lists en comparación con estos otros contenedores de secuencia es que carecen de acceso directo a los elementos por su posición; Por ejemplo, para acceder al sexto elemento de una lista, uno tiene que iterar desde una posición conocida (como el principio o el final) a esa posición, lo que toma tiempo lineal en la distancia entre estos. También consumen algo de memoria extra para mantener la información de enlace asociada a cada elemento (lo que puede ser un factor importante para grandes listas de elementos de tamaño pequeño).

# **Listas en la STL** ( Lists in STL) | Introducción

**Member functions**

| | |
|---|---|
| (constructor) | Construct list (public member function) |
| (destructor) | List destructor (public member function) |
| operator= | Assign content (public member function) |

**Iterators:**

| | |
|---|---|
| begin | Return iterator to beginning (public member function) |
| end | Return iterator to end (public member function) |
| rbegin | Return reverse iterator to reverse beginning (public member function) |
| rend | Return reverse iterator to reverse end (public member function) |
| cbegin | Return const_iterator to beginning (public member function) |
| cend | Return const_iterator to end (public member function) |
| crbegin | Return const_reverse_iterator to reverse beginning (public member function) |
| crend | Return const_reverse_iterator to reverse end (public member function) |

Capacity:

| | |
|---|---|
| empty | Test whether container is empty (public member function) |
| size | Return size (public member function) |
| max_size | Return maximum size (public member function) |

Element access:

| | |
|---|---|
| front | Access first element (public member function) |
| back | Access last element (public member function) |

et allocator (public member functio

# **Listas en la STL** ( Lists in STL) | Introducción

Modifiers:

| | |
|---|---|
| assign | Assign new content to container (public member function) |
| emplace_front | Construct and insert element at beginning (public member function) |
| push_front | Insert element at beginning (public member function) |
| pop_front | Delete first element (public member function) |
| emplace_back | Construct and insert element at the end (public member function) |
| push_back | Add element at the end (public member function) |
| pop_back | Delete last element (public member function) |
| emplace | Construct and insert element (public member function) |
| insert | Insert elements (public member function) |
| erase | Erase elements (public member function) |
| swap | Swap content (public member function) |
| resize | Change size (public member function) |
| clear | Clear content (public member function) |

Operations:

| | |
|---|---|
| splice | Transfer elements from list to list (public member function) |
| remove | Remove elements with specific value (public member function) |
| remove_if | Remove elements fulfilling condition (public member function template) |
| unique | Remove duplicate values (public member function) |
| merge | Merge sorted lists (public member function) |
| sort | Sort elements in container (public member function) |
| reverse | Reverse the order of elements (public member function) |

Observers:

| | |
|---|---|
| get_allocator | Get allocator (public member functio |



# **Listas en la STL** ( Lists in STL) | Introducción

```cpp
// CPP program to show the implementation of List
#include <iostream>
#include <iterator>
#include <list>
using namespace std;

// function for printing the elements in a list
void showlist(list<int> g)
{
    list<int>::iterator it;
    for (it = g.begin(); it != g.end(); ++it)
        cout << '\t' << *it;
    cout << '\n';
}

// Driver Code
int main()
{

    list<int> gqlist1, gqlist2;

    for (int i = 0; i < 10; ++i) {
        gqlist1.push_back(i * 2);
        gqlist2.push_front(i * 3);
    }
    cout << "\nList 1 (gqlist1) is : ";
    showlist(gqlist1);

    cout << "\nList 2 (gqlist2) is : ";
    showlist(gqlist2);

    //front()   Returns the value of the first element in the list.
    cout << "\ngqlist1.front() : " << gqlist1.front();
    //back()    Returns the value of the last element in the list.
    cout << "\ngqlist1.back() : " << gqlist1.back();

    cout << "\ngqlist1.pop_front() : ";
    //pop_front()   Removes the first element of the list, and reduces size of the list by 1.
    gqlist1.pop_front();
    showlist(gqlist1);

    cout << "\ngqlist2.pop_back() : ";
    //pop_back()    Removes the last element of the list, and reduces size of the list by 1.
    gqlist2.pop_back();
    showlist(gqlist2);

    cout << "\ngqlist1.reverse() : ";
    //reverse() Reverses the list.
    gqlist1.reverse();
    showlist(gqlist1);

    cout << "\ngqlist2.sort(): ";
    //sort()    Sorts the list in increasing order.
    gqlist2.sort();
    showlist(gqlist2);

    return 0;
}
```



# Observaciones finales

## Sobre Estructuras de Datos, Listas, Matrices, Vectores…

Se han introducido listas vinculadas para superar las limitaciones de las matrices al permitir la asignación dinámica de las cantidades necesarias de memoria. Además, las listas vinculadas permiten una fácil inserción y eliminación de información, ya que tales operaciones tienen un impacto local en la lista. Para insertar un nuevo elemento al principio de una matriz, todos los elementos de la matriz deben desplazarse para dejar espacio para el nuevo elemento; por lo tanto, la inserción tiene un impacto global en la matriz. La eliminación es la misma. Entonces, ¿deberíamos usar siempre listas vinculadas en lugar de matrices? Las matrices tienen algunas ventajas sobre las listas vinculadas, a saber, que permiten el acceso aleatorio. Para acceder al décimo nodo de una lista vinculada, se deben pasar los nueve nodos anteriores. En la matriz, podemos ir directamente a la décima celda. Por lo tanto, si es necesario un acceso inmediato de cualquier elemento, entonces una matriz es una mejor opción. Este fue el caso con la búsqueda binaria, y será el caso con la mayoría de los algoritmos de clasificación . Pero si estamos constantemente accediendo solo a algunos elementos, el primero, el segundo, el último y similares, y si cambiar la estructura es el núcleo de un algoritmo, entonces usar una lista vinculada es una mejor opción. Un buen ejemplo es una cola, que se discute en el siguiente unidad. Otra ventaja en el uso de matrices es el espacio. Para mantener los elementos en matrices, las celdas deben ser del tamaño de los elementos. En las listas vinculadas, almacenamos un elemento por nodo, y el nodo también incluye al menos un puntero; en listas doblemente vinculadas, el nodo contiene dos punteros. Para listas vinculadas grandes, se necesita una cantidad significativa de memoria para almacenar los punteros. Por lo tanto, si un problema no requiere muchos cambios de datos, entonces tener una matriz de gran tamaño puede no ser un desperdicio en absoluto si su tamaño se compara con la cantidad de espacio necesario para que la estructura vinculada almacene los mismos datos que la matriz.

Muchas Gracias.